

C-MAPSS Operating Conditions

Saxena et al., PHM'08 — Sub-datasets FD002 and FD004 — *Working reference*

Contents

1	What Are Operating Conditions?	1
2	The Throttle Resolver Angle (TRA)	1
3	The Six Conditions	2
3.1	Chart A — Altitude vs Mach number	2
3.2	Chart B — Altitude vs TRA	3
3.3	Chart C — Mach number vs TRA	4
3.4	Chart D — Centroid reference table	5
4	Why Only FD002 and FD004?	5
5	How the Six Conditions Are Recovered	5
6	What Is a Centroid, and Why Does It Matter Here?	6
6.1	The general idea	6
6.2	Why C-MAPSS centroids are exact	6
6.3	The four algorithm steps	6
6.4	Why the label numbers seem random	7
7	Why Operating Conditions Matter for Analysis	7

What Are Operating Conditions?

A turbofan engine does not always operate at the same altitude, speed, and throttle setting. During a single flight it passes through multiple distinct phases — takeoff, climb, cruise, and descent. Each phase demands different engine behaviour and produces different sensor readings.

In C-MAPSS, an **operating condition** is one specific combination of three variables:

Column	Physical quantity	Description
op_setting_1	Altitude	Height above sea level. Stored as <i>thousands of feet</i> in the file. Range: 0 to 42,000 ft.
op_setting_2	Mach number	Flight speed as a fraction of the speed of sound. Dimensionless. Range: 0 to 0.84.
op_setting_3	TRA	Throttle Resolver Angle, in degrees. The physical angle of the pilot's throttle lever. Range: 20° to 100°.

The paper states the ranges but does not list the six exact combinations. They are recovered from the data by KMeans clustering on the three `op_setting` columns (see Section 5).

The Throttle Resolver Angle (TRA)

TRA is the most intuitive of the three variables. It is the physical angle of the throttle lever on the flight deck — like a car accelerator pedal, but measured as an angle rather than displacement. A resolver (rotary sensor) reads the lever position and sends it to the engine controller.

TRA	Thrust level	Meaning
0°	Idle	Engine at minimum power.
60°	Reduced	Partial thrust. Used for fuel-efficient cruise.
100°	Maximum	Full commanded thrust.

Why is TRA = 100° at high altitude? At 42,000 ft the air density is roughly 20% of sea level. The engine must work at full throttle just to maintain cruise thrust — there are simply not enough air molecules to produce adequate thrust at reduced settings. So counterintuitively, the highest altitudes require the highest TRA.

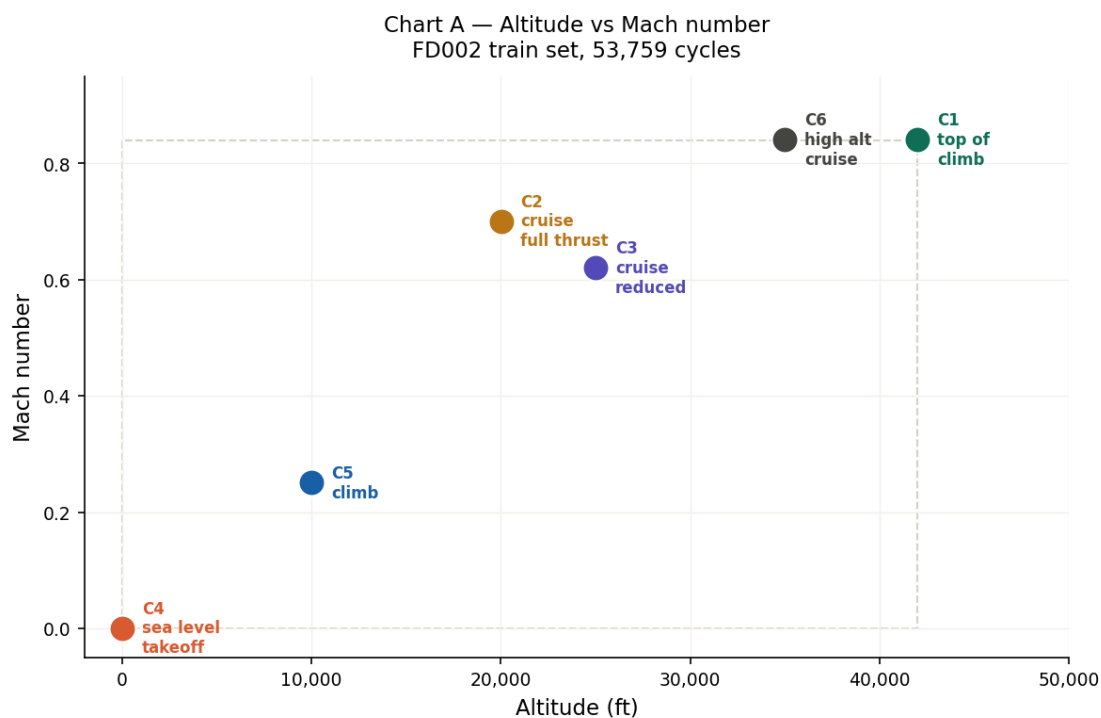
The one exception is C3 (25,000 ft, TRA = 60°) — a mid-altitude cruise phase where the aircraft can afford to reduce fuel burn without falling below minimum cruise thrust.

The Six Conditions

FD002 and FD004 simulate an engine cycling through six distinct flight conditions within each engine life. Every engine visits all six conditions — the order is random from cycle to cycle.

The following four charts examine the conditions from every angle, one variable pair at a time.

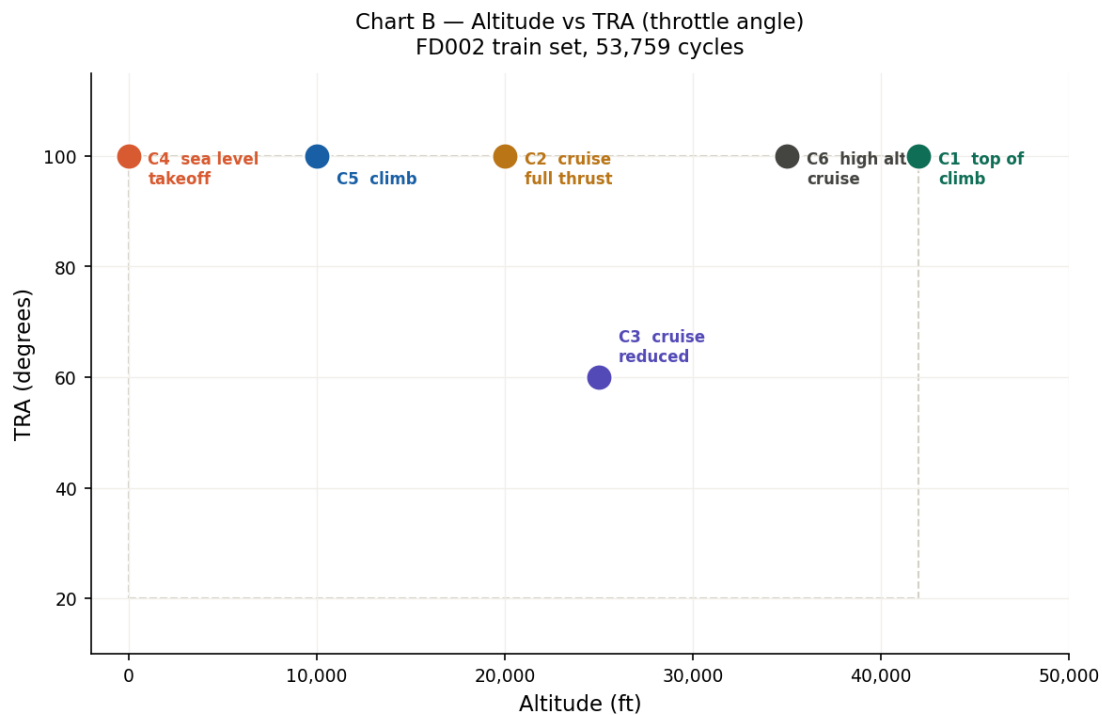
Chart A — Altitude vs Mach number



C6 and C1 share Mach=0.84 — they cannot be separated in this projection alone.

Figure 1: Altitude vs Mach number. The dashed box is the paper-stated range (alt 0–42,000 ft, Mach 0–0.84). Five of six conditions are clearly separated. C6 and C1 share Mach = 0.84 and appear close together — altitude alone separates them (35,000 vs 42,000 ft).

Reading Chart A left to right traces the flight profile: C4 at the origin (sea level, stationary), C5 at low altitude climbing, C2/C3 at mid-cruise, and C6/C1 at high altitude. The general trend is diagonal — higher altitude correlates with higher speed, as expected in aviation.

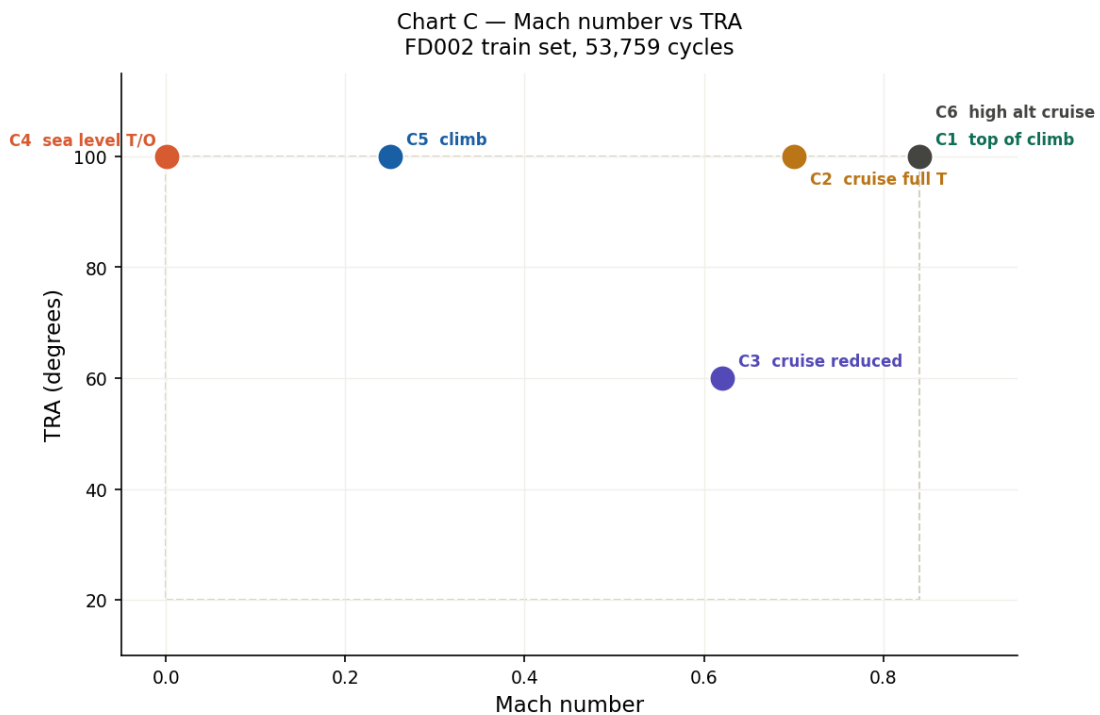
Chart B — Altitude vs TRA

C3 is the only condition at TRA=60° — all others are at full throttle (100°).

Figure 2: Altitude vs TRA (throttle resolver angle). Five conditions sit on a flat line at TRA = 100° (full throttle). C3 is the only outlier at TRA = 60°, at 25,000 ft. This is the most visually striking separation in the dataset.

Chart B directly answers the question “why is TRA almost always 100°?” The flat horizontal line at TRA = 100° shows that across sea level takeoff, climb, and all high-altitude phases, the engine is at full commanded thrust. Only C3 — the mid-altitude reduced-thrust cruise — breaks the pattern.

Chart C — Mach number vs TRA



C6 and C1 overlap here (both Mach=0.84, TRA=100°) — altitude separates them.

Figure 3: Mach number vs TRA. C6 and C1 overlap exactly (both at Mach = 0.84, TRA = 100°). In this projection they are indistinguishable. Only altitude (Chart A) separates them. This demonstrates why all three `op_setting` columns are needed for clustering.

Chart C reveals the fundamental reason why KMeans is run on all three columns simultaneously. No single 2D projection cleanly separates all six conditions. C6 and C1 are identical in Mach and TRA — altitude is the only distinguishing dimension. A 2D clustering on any two columns alone would merge them into one cluster.

Chart D — Centroid reference table

Chart D — Six flight condition centroids (FD002 / FD004)

Cond	Alt (ft)	Mach	TRA	Flight phase	Note
C4	0	0.00	100°	sea level takeoff	—
C5	10,000	0.25	100°	climb	—
C2	20,000	0.70	100°	cruise, full thrust	—
C3	25,000	0.62	60°	cruise, reduced thrust	Only condition at reduced thrust
C6	35,000	0.84	100°	high altitude cruise	Shares Mach with C1
C1	42,000	0.84	100°	top of climb	Shares Mach with C6

Figure 4: Complete centroid reference for all six conditions. The Note column highlights which conditions share a dimension with another, explaining why 3D clustering is required.

Why Only FD002 and FD004?

FD001 and FD003 simulate a single operating condition — sea level only. All rows have `op_setting_1 = 0`, `op_setting_2 = 0`, `op_setting_3 = 100`.

This makes them simpler — sensor readings are directly comparable across cycles without any condition-wise normalisation. The `condition_id = 1` for every row and carries no information.

How the Six Conditions Are Recovered

The six exact combinations are recovered by KMeans clustering ($k = 6$) on the three `op_setting` columns. The algorithm minimises:

$$J = \sum_{i=1}^n \|x_i - \mu_{c(i)}\|^2 \quad (1)$$

where $x_i = (\text{op_setting_1}_i, \text{op_setting_2}_i, \text{op_setting_3}_i)$ and μ_k is the centroid of cluster k .

Because the simulator snaps to exactly six discrete flight points (with only tiny noise), inertia is near zero ($J \approx 0.41$ for FD002). The centroid of each cluster is the exact flight point, not an approximation.

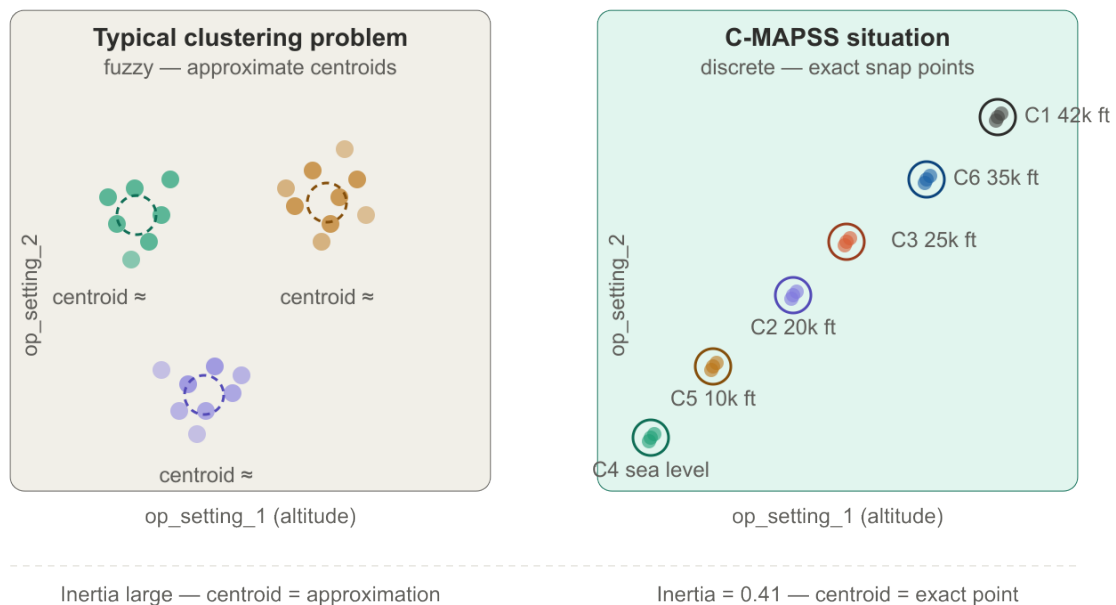
Label ordering warning. Condition labels (C1–C6) are assigned by KMeans in arbitrary order. The same physical condition may receive a different label in FD002 vs FD004. Always compare conditions by `op_setting` values, never by label number.

What Is a Centroid, and Why Does It Matter Here?

The general idea

A **centroid** is the centre of a cluster — the point that minimises the average distance to all members of that cluster. Geometrically it is the mean position of all points assigned to the cluster.

In a typical clustering problem, data points are spread in fuzzy clouds. The centroid sits somewhere inside the cloud — it is an approximation, not an actual data point. Figure 5 shows the contrast between the typical case and what happens in C-MAPSS.



KMeans does not approximate here — it recovers exact flight points

Figure 5: Left: typical clustering — data spreads in fuzzy clouds, centroids are approximations (shown with dashed circles). Right: C-MAPSS — all data snaps to exactly 6 discrete flight points. The centroid coincides exactly with the data point. Inertia = 0.41 (near zero) confirms this.

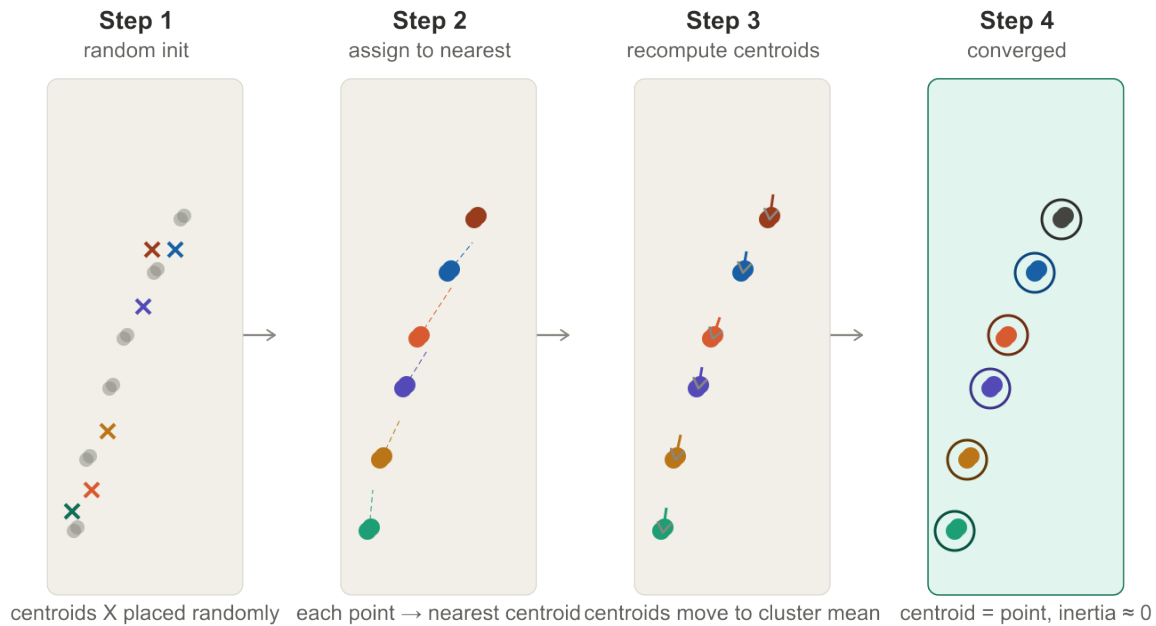
Why C-MAPSS centroids are exact

The C-MAPSS simulator always sets the engine to one of exactly six predetermined flight conditions before recording each cycle. Every row in the dataset therefore has `op_setting` values that are essentially identical to one of the six target points — with only tiny sensor noise (± 0.001 or less).

When KMeans clusters these points, each cluster contains thousands of rows that all sit at almost exactly the same 3D coordinate. The mean of those rows — the centroid — lands at that coordinate. There is no approximation because there is no spread to approximate.

The four algorithm steps

Figure 6 shows what KMeans actually does on this data, step by step.



Because C-MAPSS snaps to 6 exact flight points, each cluster has near-zero spread.
 KMeans converges in few iterations. The centroid IS the flight point — not an approximation.
 This is why inertia = 0.41 (near zero) and every engine sees all 6 conditions exactly.

Figure 6: KMeans in four steps on C-MAPSS data. Step 1: place 6 centroids randomly (X marks). Step 2: assign each data point to its nearest centroid (color = assigned cluster, dashed lines show assignment). Step 3: recompute each centroid as the mean of its cluster (arrows show centroids moving toward the true flight points). Step 4: converged — centroid sits exactly on the data point, inertia ≈ 0.

The key observation from Step 4: the centroid ring and the data dot coincide. This is not a coincidence — it is a consequence of the discrete simulator design. In a real-world vibration or wear dataset, you would never see this. The C-MAPSS data is unusually clean precisely because it is a digital twin, not real sensor data.

Why the label numbers seem random

Looking at Chart D, the conditions sorted by altitude are C4, C5, C2, C3, C6, C1 — not C1, C2, C3, C4, C5, C6. This is because KMeans assigns label numbers by the order in which centroids happened to converge in that particular run, not by altitude or any physical property.

The label number has no physical meaning. C1 does not mean “lowest altitude” or “first condition”. It simply means “whichever cluster was internally labelled 0 by the KMeans run on that sub-dataset”. Two sub-datasets run independently (FD002 and FD004) will produce different label assignments for the same physical conditions. Always trace a condition back to its `op_setting` values to know what it physically represents.

Why Operating Conditions Matter for Analysis

Raw sensor values are not comparable across conditions. Fan inlet temperature T_2 at sea level is hundreds of degrees Rankine higher than at 42,000 ft — purely due to altitude, not engine health. Feeding raw values into a model trains it to recognise flight conditions rather than degradation.

The solution is condition-wise normalisation:

$$x_{\text{norm}} = \frac{x - \mu_{s,c}}{\sigma_{s,c}} \quad (2)$$

where $\mu_{s,c}$ and $\sigma_{s,c}$ are computed from healthy (early-life) cycles at condition c only. After this step each value expresses deviation from the healthy baseline at that specific flight condition.

Summary.

- FD001/FD003: one condition, global normalisation sufficient.
- FD002/FD004: six conditions, condition-wise normalisation required.
- `condition_id` identifies the condition per cycle.
- No 2D projection separates all six conditions — all three `op_setting` columns are needed.
- Labels are not comparable across sub-datasets.